

# C语言程序设计 — 期末总复习手册

覆盖第1~8章全部核心考点 + 错题复盘 + 输入输出速查手册  
适用于期末考试复习，可导出 PDF 打印

## 目录

- 1. 第1章 程序设计和C语言
- 2. 第2章 算法——程序的灵魂
- 3. 第3章 顺序程序设计
- 4. 第4章 选择结构程序设计
- 5. 第5章 循环结构程序设计
- 6. 第6章 利用数组处理批量数据
- 7. 第7章 用函数实现模块化程序设计
- 8. 第8章 善于利用指针
- 9. 附录：C语言输入输出速查手册

## 第1章 程序设计和C语言

### 1.1 程序与程序设计

| 概念     | 定义                       |
|--------|--------------------------|
| 程序     | 计算机能够识别和执行的一组指令（有序的指令集合） |
| 程序设计   | 设计、编制、调试程序的过程            |
| 程序设计语言 | 人与计算机交流的规则化语言            |

### 1.2 计算机语言的发展

机器语言 → 汇编语言 → 高级语言  
(二进制)      (助记符)      (接近自然语言)

- **机器语言**：计算机唯一可直接执行的语言，但难以记忆和使用
- **汇编语言**：用助记符代替二进制，需汇编程序翻译
- **高级语言**：接近人类自然语言和数学语言，需编译程序翻译，可移植性好

### 1.3 C语言的发展与特点

- **诞生**：1972年，贝尔实验室，Dennis Ritchie 设计
- **ANSI C 标准**：1983年制定，1989年通过（ANSI X3.159-1989）

- **ISO C 标准**：1990年，国际标准化组织采纳（ISO/IEC 9899:1990）
- **C99/C11/C17**：后续标准更新

**C语言的主要特点**（★常考简答）：

1. 语言简洁、紧凑，使用方便灵活（仅37个关键字）
2. 运算符丰富（34种运算符）
3. 数据类型丰富（整型、浮点型、字符型、指针、结构体等）
4. 具有结构化的控制语句
5. 语法限制不太严格，程序设计自由度大
6. 允许直接访问物理地址，能进行位操作
7. 生成目标代码质量高，程序执行效率高
8. 可移植性好

**1.4 简单C语言程序的结构**（★核心必考）

```
#include <stdio.h>           // 编译预处理指令（头文件）

int main()                   // 主函数（程序入口）
{
    printf("Hello World\n"); // 函数体（语句）
    return 0;                // 返回值
}
```

**程序结构要点：**

1. 一个C程序**有且仅有一个** `main` 函数
2. 程序从 `main` 函数开始执行，`main` 函数执行完毕程序结束
3. `#include` 是**编译预处理指令**，不是语句（不用加分号）
4. 每个语句以 `;` 结束
5. `/* ... */` 为块注释，`//` 为行注释
6. C语言本身不提供输入输出语句，由库函数（如 `printf`、`scanf`）实现

**1.5 C程序的开发流程**（★常考填空）

编辑(.c源文件) → 编译(.obj目标文件) → 连接(.exe可执行文件) → 运行

| 阶段 | 产物         | 说明             |
|----|------------|----------------|
| 编辑 | .c 源文件     | 编写C源代码         |
| 编译 | .obj 目标文件  | 编译器将源程序翻译成机器指令 |
| 连接 | .exe 可执行文件 | 将目标文件与库函数等连接装配 |
| 运行 | 输出结果       | 执行程序           |

## 1.6 本章习题

### 题号 题型

|     |                           |
|-----|---------------------------|
| 1-1 | 概念题：什么是程序？什么是程序设计？        |
| 1-2 | 概念题：为什么需要计算机语言？高级语言特点？    |
| 1-3 | 概念题：源程序/目标程序/可执行程序等名词解释   |
| 1-4 | 编程：输出 Hello World!        |
| 1-5 | 编程：输出右斜星号图形               |
| 1-6 | 编程：输入三个值输出最大值             |
| 1-7 | 操作题：VS 2010 编辑/编译/连接/运行流程 |

## 1.7 高频考点

### 🔥 考点1：C程序的基本结构

- "一个C程序可以有多个main函数" — ❌ 只能有一个
- "C程序从第一个函数开始执行" — ❌ 从main函数开始
- "分号是语句的结束标志" — ✅
- "预处理指令必须以 # 开头" — ✅

### 🔥 考点2：程序开发流程顺序

- 口诀：编-译-连-运（编辑→编译→连接→运行）

### 🔥 考点3：头文件与预处理

- `#include <stdio.h>` 的作用：包含标准输入输出库函数声明
- 不写 `#include` 能否使用 `printf`？理论上某些编译器会警告但可能通过，标准要求必须包含

### 🔥 考点4：C语言的特点

- C语言属于高级语言，但兼具低级语言功能（可直接访问地址）

## 1.8 易错点警示

| 易错点        | 错误示例                                      | 正确做法                                             |
|------------|-------------------------------------------|--------------------------------------------------|
| 预处理指令加分号   | <code>#include &lt;stdio.h&gt;;</code>    | <code>#include &lt;stdio.h&gt;</code>            |
| main函数拼写错误 | <code>mian()</code> / <code>Main()</code> | <code>main()</code>                              |
| 忘记返回值      | <code>void main()</code>                  | <code>int main()</code> 并 <code>return 0;</code> |
| 中文符号       | <code>printf("Hello");</code>             | <code>printf("Hello");</code> （全英文符号）            |
| 注释嵌套       | <code>/* a /* b */ c */</code>            | C语言注释不能嵌套                                        |
| 漏写分号       | <code>printf("Hello")</code>              | <code>printf("Hello");</code>                    |

## 第2章 算法——程序的灵魂

### 2.1 算法的概念（★必考）

| 概念 | 定义                        |
|----|---------------------------|
| 算法 | 为解决一个问题而采取的方法和步骤，是程序的灵魂   |
| 程序 | 数据结构 + 算法 + 程序设计方法 + 语言工具 |

通俗理解：算法 = 做事的步骤。比如菜谱、洗衣机流程、乐谱等。

### 2.2 算法的五大特性（★高频简答）

- 1. **有穷性**：有限步骤后结束，每一步在有限时间内完成
- 2. **确定性**：每一步都有确切的含义，不能有二义性
- 3. **有零个或多个输入**：从外界获取必要信息
- 4. **有一个或多个输出**：必须产生结果
- 5. **有效性**：每一步都能被有效执行，能得到确定结果

### 2.3 算法的表示方法（★常考选择题）

| 方法      | 说明          | 特点          |
|---------|-------------|-------------|
| 自然语言    | 用日常语言描述     | 通俗但易歧义      |
| 传统流程图   | 用图形符号表示算法   | 直观形象，但占用篇幅大 |
| N-S 流程图 | 盒图，去掉流程线    | 结构化，紧凑      |
| 伪代码     | 介于自然语言和代码之间 | 灵活，接近程序     |
| 计算机语言   | 用 C 等语言实现   | 计算机可直接执行    |

流程图常用符号：

| 符号    | 名称     | 含义      |
|-------|--------|---------|
| 圆角矩形  | 起止框    | 算法开始/结束 |
| 矩形    | 处理框    | 计算/赋值   |
| 菱形    | 判断框    | 条件判断    |
| 箭头    | 流程线    | 执行方向    |
| 平行四边形 | 输入/输出框 | 数据读写    |

### 2.4 三种基本结构（★核心必考）

| 结构   | 特点                |
|------|-------------------|
| 顺序结构 | 按先后顺序依次执行，无分支、无跳转 |
| 选择结构 | 根据条件判断，选择执行某个分支   |
| 循环结构 | 在满足条件时重复执行某段代码    |

共同特点（★简答常考）：

- 只有一个入口
- 只有一个出口
- 结构内每一部分都有机会被执行到
- 结构内不存在"死循环"

由这三种基本结构组成的算法属于 **结构化算法**。

## 2.5 结构化程序设计方法（★简答常考）

**核心思想**：自顶向下、逐步细化、模块化设计、结构化编码

| 内容    | 说明           |
|-------|--------------|
| 自顶向下  | 从总体目标出发，逐层分解 |
| 逐步细化  | 将大问题逐步分解为小问题 |
| 模块化设计 | 将程序分为独立功能模块  |
| 结构化编码 | 仅用三种基本结构编写代码 |

## 2.6 本章习题

| 题号  | 题型                                 |
|-----|------------------------------------|
| 2-1 | 概念题：什么是算法？举3个生活例子                  |
| 2-2 | 概念题：什么是结构化算法？为什么要提倡？               |
| 2-3 | 概念题：三种基本结构特点 + 设计两种新结构             |
| 2-4 | 流程图：8小题（交换/最大数/排序/累加/整除/素数/公约数/求根） |
| 2-5 | N-S 图：为第4题各题画 N-S 图                |
| 2-6 | 伪代码：为第4题各题写伪代码                     |
| 2-7 | 概念题：什么叫结构化程序设计？主要内容？               |
| 2-8 | 设计题：自顶向下逐步细化（闰年/求根/最大数）            |

## 2.7 高频考点

🔥 考点1：算法的五大特性

- **题型：**简答/选择
- **口诀：**有穷、确定、有效、零多输入、一多输出

 **考点2：三种基本结构**

- 传统流程图有流程线（允许任意跳转），N-S 图没有流程线（强制结构化）

 **考点3：流程图 → 代码**

- 常考：判断素数、求最大公约数（辗转相除法）、求闰年

 **考点4：经典算法**

| 算法                 | 题型热度 |
|--------------------|------|
| 素数判断               | ☆☆☆  |
| 最大公约数（辗转相除）        | ☆☆☆  |
| 求根公式 $ax^2+bx+c=0$ | ☆☆   |
| 闰年判断               | ☆☆   |
| 交换两个变量的值           | ☆☆☆  |

2.8 易错点警示

| 易错点            | 说明                     |
|----------------|------------------------|
| 算法 ≠ 程序        | 算法是思路，程序是算法的具体实现       |
| 有穷性 ≠ 死循环      | 循环必须有退出条件              |
| 基本结构特点         | 必须"单入口单出口"             |
| 流程图菱形框         | 必须有两个出口（T/F），不能只有一个    |
| N-S 图 vs 传统流程图 | N-S 图无流程线，传统流程图有箭头和流向线 |

第3章 顺序程序设计

3.1 数据的表现形式

常量（ 常考）

| 类型   | 示例                | 说明           |
|------|-------------------|--------------|
| 整型常量 | 100, -5, 0        | 不带小数点的整数     |
| 实型常量 | 3.14, -2.5, 1.2e3 | 带小数点或指数形式    |
| 字符常量 | 'A', '6', '\n'    | 用单引号括起来的单个字符 |

| 类型    | 示例                             | 说明                          |
|-------|--------------------------------|-----------------------------|
| 字符串常量 | "Hello", "a"                   | 用双引号括起来，自动加 <code>\0</code> |
| 符号常量  | <code>#define PI 3.1416</code> | 用标识符代表常量                    |

字符 'a' 与字符串 "a" 的区别 (★高频):

|    | 'a'               | "a"                       |
|----|-------------------|---------------------------|
| 长度 | 1 字节              | 2 字节 (含 <code>\0</code> ) |
| 类型 | <code>char</code> | <code>char[]</code>       |

## 变量

- **定义**: 程序运行期间值可以改变的量
- **先定义, 后使用**
- **变量名**: 字母/下划线开头, 字母+数字+下划线组成, 区分大小写

## 3.2 数据类型 (★核心必考)

数据类型

- ├ 基本类型
  - ├ 整型 `int` (2或4字节, -32768~32767 或  $-2^{31} \sim 2^{31}-1$ )
  - ├ 字符型 `char` (1字节, -128~127)
  - ├ 单精度浮点型 `float` (4字节, 6~7位有效数字)
  - └ 双精度浮点型 `double` (8字节, 15~16位有效数字)
- ├ 构造类型 (数组、结构体、共用体、枚举)
- ├ 指针类型
- └ 空类型 `void`

## 3.3 运算符与表达式 (★重点)

### 算术运算符

| 运算符                                          | 含义     | 优先级 |
|----------------------------------------------|--------|-----|
| <code>*</code> <code>/</code> <code>%</code> | 乘、除、取余 | 高   |
| <code>+</code> <code>-</code>                | 加、减    | 低   |

- `%` 两侧必须为**整型**
- 整数除整数结果仍为整数 (截断小数):  $5/3 = 1$

### 自增自减

| 形式               | 含义       |
|------------------|----------|
| <code>++i</code> | 先加1, 后使用 |

## 形式 含义

`i++` 先使用，后加1

## 赋值运算符与类型转换

- `=` 赋值，`+=` `--` `*=` `/=` `%=` 复合赋值
- 自动类型转换：`int`  $\rightarrow$  `float`  $\rightarrow$  `double`（向高精度转）
- 强制类型转换：`(类型名)表达式`，如 `(int)3.14`

## 3.4 输入输出函数（★必考）

### printf — 格式输出

```
printf("格式控制", 输出列表);
```

| 格式符             | 含义       | 格式符              | 含义                        |
|-----------------|----------|------------------|---------------------------|
| <code>%d</code> | 有符号十进制整数 | <code>%f</code>  | 单精度浮点数                    |
| <code>%c</code> | 单个字符     | <code>%lf</code> | 双精度（推荐用 <code>%f</code> ） |
| <code>%s</code> | 字符串      | <code>%e</code>  | 指数形式                      |
| <code>%o</code> | 八进制整数    | <code>%x</code>  | 十六进制整数                    |
| <code>%u</code> | 无符号十进制整数 | <code>%%</code>  | 输出百分号本身                   |

### 附加格式：

- `%5d` — 宽度5，右对齐
- `%.2f` — 保留2位小数
- `%-5d` — 左对齐
- `%05d` — 不足5位用0填充

### scanf — 格式输入

```
scanf("格式控制", 地址列表); // ⚠ 变量前必须加 &
```

| 关键点   | 说明                                                         |
|-------|------------------------------------------------------------|
| & 地址符 | <code>scanf("%d", &amp;a);</code> 必须加 <code>&amp;</code>   |
| 分隔符   | 空格、Tab、回车均可作为输入分隔                                          |
| 格式匹配  | <code>scanf("a=%d", &amp;a)</code> 输入时必须写 <code>a=3</code> |

### putchar / getchar — 字符输入输出

```
char c;
c = getchar();    // 从键盘读一个字符
putchar(c);       // 输出一个字符
```

## 3.5 本章习题

| 题号  | 题型                           |
|-----|------------------------------|
| 3-1 | 编程：计算 GDP 年增长率 $p=(1+r)^n$   |
| 3-2 | 编程：存款利息 5 种方案比较              |
| 3-3 | 编程：房贷还清月数计算（含对数）             |
| 3-4 | 读程序：char 与 ASCII 码输出分析       |
| 3-5 | scanf 输入格式练习                 |
| 3-6 | 编程：China→Glmre 密码翻译          |
| 3-7 | 编程：圆/圆柱/圆球几何计算               |
| 3-8 | 编程：getchar/putchar 练习 + 概念思考 |

## 3.6 本章错题复盘

### ⚠ 3-4(2) char 溢出

**错误：**char c1=197; printf("%d",c1) 输出 197

**正确：**197 超 signed char 范围(-128~127)，手算过程如下：

197 的 8 位二进制： 11000101  
 ↓ 在 signed char 中，最高位为 1 → 判定为负数（计算机存的是补码）  
 补码 11000101 → 减1得反码 11000100  
 反码 11000100 → 符号位不变、其余取反 → 原码 10111011  
 原码 10111011 → 后7位 0111011 = 32+16+8+2+1 = 59  
 加上符号位：-59

### ⚠ 3-5 scanf %c 陷阱

**错误：**a=3b=7 8.5 71.82 A a（数字和字符间有空格）

**正确：**a=3b=7 8.5 71.82Aa（字符紧跟数字后，%c 不跳空格）

## 3.7 高频考点

### 🔥 考点1：printf 格式符

- **题型：**读程序写结果
- printf("%c %d", 97, 'a'); → 输出 a 97

🔥 考点2：scanf 取地址符

- 易错：scanf("%d", a); ❌ → scanf("%d", &a); ✅

🔥 考点3：char 与 int 的关系

- char 存储的是 ASCII 码值（整数），char 和 int 在一定范围内可互换
- 'A' = 65, 'a' = 97, '0' = 48
- char 范围 -128~127，超过127会溢出

🔥 考点4：整数除法截断

- 5/3 = 1（不是1.666）
- 1/2 = 0（整数除法，结果为0）

🔥 考点5：++i 和 i++ 的区别

- 单独的 i++; 和 ++i; 等价
- 参与表达式时不同：a = i++ vs a = ++i

3.8 易错点警示

| 易错点                 | 错误                | 正确                      |
|---------------------|-------------------|-------------------------|
| scanf 忘写 &          | scanf("%d", a);   | scanf("%d", &a);        |
| 取余用浮点数              | 5.0 % 2           | % 只能用于整数                |
| 整数除整数               | 5/3 得 1           | 5.0/3 才得 1.666...       |
| 字符常量/字符串混淆          | "A" 当字符用          | 字符用 'A'，字符串用 "A"        |
| scanf 格式多余字符        | scanf("a=%d", &a) | 输入时必须原样输入 a=3           |
| char 溢出             | c1=197 char 存不下   | char 范围 -128~127，改用 int |
| printf 打印 char 用 %d | 把字符当整数输出          | 输出 ASCII 码用 %d，输出字符用 %c |

第4章 选择结构程序设计

4.1 关系运算符与关系表达式

| 运算符       | 含义              | 优先级 |
|-----------|-----------------|-----|
| < <= > >= | 小于、小于等于、大于、大于等于 | 高   |
| == !=     | 等于、不等于          | 低   |

- 关系表达式的结果：真=1，假=0
- 易错：== 是等于，= 是赋值，千万别写错！

4.2 逻辑运算符与逻辑表达式（★ 必考）

| 运算符 | 含义        | 优先级 | 结合性 |
|-----|-----------|-----|-----|
| !   | 逻辑非 (NOT) | 最高  | 右结合 |
| &&  | 逻辑与 (AND) | 中   | 左结合 |
|     | 逻辑或 (OR)  | 最低  | 左结合 |

**优先级总排序：**单目（含 `! ++ --` 等）> 算术 > 关系 > `&&` > `||` > 赋值

⚠ 逻辑非 `!` 属于**单目运算符**，优先级高于所有算术运算符。`!a + b` 等价于 `(!a) + b`，而非 `!(a + b)`。

**短路特性**（★高频）：

- `a && b`：若 `a` 为假，不再计算 `b`
- `a || b`：若 `a` 为真，不再计算 `b`

**真假的判断：**C语言中，非0即真，0为假

## 4.3 if 语句（★核心）

三种形式

```
// ① 单分支
if (条件) 语句;

// ② 双分支
if (条件) 语句1;
else 语句2;

// ③ 多分支
if (条件1) 语句1;
else if (条件2) 语句2;
else if (条件3) 语句3;
else 语句n;
```

**if-else 配对规则**（★坑点）

**else 总是与前面最近的未配对的 if 配对！**

```
if (a > 0)
    if (b > 0)
        c = 1;
    else // ← 这个 else 和哪个 if 配对?
        c = -1; // 答案：和 if(b>0) 配对，而非 if(a>0)
```

加 `{ }` 可以改变配对关系，也是好习惯。

## 4.4 switch 语句

```
switch (整型表达式)    // ⚠️ 只能是整型/字符型，不能是浮点型
{
    case 常量1: 语句1; break;
    case 常量2: 语句2; break;
    default:    语句n; break;
}
```

| 要点         | 说明                                  |
|------------|-------------------------------------|
| switch 表达式 | 必须为 <b>整型或字符型</b> ，不能是 float/double |
| case 后     | 必须为 <b>常量表达式</b> ，不能是变量             |
| break      | 跳出 switch，若漏写会 <b>穿透</b> 到下一个 case  |
| default    | 可放在任意位置，习惯放最后                       |

## 4.5 条件运算符（三目）

```
max = (a > b) ? a : b;    // 若 a>b 为真取 a，否则取 b
// 表达式1 ? 表达式2 : 表达式3
```

- 结合性：**右结合**
- 优先级：仅高于赋值和逗号运算符

## 4.6 本章习题

| 题号   | 题型                            |
|------|-------------------------------|
| 4-1  | 概念：算术运算/关系运算/逻辑运算的区别          |
| 4-2  | 概念：C语言中"真"和"假"的表示与判断          |
| 4-3  | 计算：5个逻辑表达式的值 (a=3,b=4,c=5)    |
| 4-4  | 编程：输入3个整数，输出最大值               |
| 4-5  | 编程：输入<1000正数，输出平方根整数部分        |
| 4-6  | 编程：分段函数（三区间）                  |
| 4-7  | 读程序：两段代码的if-else配对与逻辑分析       |
| 4-8  | 编程：百分制→等级 A/B/C/D/E           |
| 4-9  | 编程：正整数求位数/逐位输出/逆序输出           |
| 4-10 | 编程：企业奖金利润提成（if + switch 两种写法） |

## 题号 题型

4-11 编程：输入4个整数，由小到大输出

4-12 编程：4个圆塔判断建筑高度

## 4.7 本章错题复盘

### ⚠ 4-3(4) 短路求值

错误：没考虑 `&&` 短路，以为 `y=b` 会执行

正确：`!(x=a)` 得 0，`&&` 左侧为假，右侧 `!(y=b) && 0` 不执行，`y` 未被赋值

### ⚠ 4-10 printf 格式

错误：`printf("%d", p)` — `p` 是 float

正确：`printf("%.2f", p)`

### ⚠ 4-12 距离公式

错误：`(y-2) + y - 2` 写成了加法

正确：`(y-2) * (y-2)` 距离公式是平方

## 4.8 高频考点

### 🔥 考点1：if-else 配对问题

程序读结果题必出，嵌套 if 的 else 归属判断。

**口诀：**else 找最近没配对的 if

### 🔥 考点2：逻辑表达式求值 + 短路

- `a=3, b=4, c=5; a+b>c && b==c → 7>5 && 4==5 → 1 && 0 → 0`
- `!(a>b) && !c || 1 → 注意 || 1 直接为真`
- `a || b+c && b-c → && 优先级高于 || → 3 || (9 && -1) → 3 || 1 → 1`

### 🔥 考点3：switch 漏写 break

- 漏 break 会导致"穿透"，连续执行多个 case

### 🔥 考点4：分段函数编程

- 核心题型：用 if-else 或 switch 实现分段函数
- printf/scanf 格式要与输入输出一致

## 4.9 易错点警示

| 易错点       | 错误                           | 正确                       |
|-----------|------------------------------|--------------------------|
| = 与 == 混淆 | <code>if (a = 3)</code> 永远为真 | <code>if (a == 3)</code> |

| 易错点         | 错误                                                            | 正确                                           |
|-------------|---------------------------------------------------------------|----------------------------------------------|
| else 配对错    | 缩进误导                                                          | else 找最近未配对的 if                              |
| switch 用浮点数 | <code>switch(2.5)</code>                                      | switch 只能用整型/字符型                             |
| case 后用变量   | <code>case a:</code>                                          | case 只能用常量                                   |
| 漏写 break    | 穿透执行                                                          | 需要的时候必须写 break                               |
| 逻辑优先级       | <code>!a &amp;&amp; b</code> 以为是 <code>!(a&amp;&amp;b)</code> | <code>!</code> 优先级高于 <code>&amp;&amp;</code> |
| 短路求值        | 误以为右侧必执行                                                      | <code>&amp;&amp;</code> 左侧为假右侧不执行            |

## 第5章 循环结构程序设计

### 5.1 三种循环语句（★核心必考）

while 循环（先判断，后执行）

```
while (条件表达式)
{
    循环体;
}
// 可能一次都不执行
```

do...while 循环（先执行，后判断）

```
do
{
    循环体;
} while (条件表达式);
// ⚠️ 分号不能丢！至少执行一次
```

for 循环（最常用）

```
for (表达式1; 表达式2; 表达式3)
{
    循环体;
}
// 表达式1: 初始 → 表达式2: 条件 → 循环体 → 表达式3: 步进
```

| 对比     | while | do-while | for |
|--------|-------|----------|-----|
| 最少执行次数 | 0次    | 1次       | 0次  |

| 对比   | while  | do-while | for    |
|------|--------|----------|--------|
| 适用场景 | 未知循环次数 | 至少执行一次   | 已知循环次数 |

经典用法对比

```
// while - 辗转相除法（未知迭代次数）
while (b != 0) { t = a % b; a = b; b = t; }

// do-while - 牛顿迭代法（至少迭代一次）
do { x0 = x1; x1 = (x0 + a/x0) / 2; } while (fabs(x1 - x0) > 1e-5);

// for - 阶乘累加（已知20次）
for (i = 1; i <= 20; i++) { temp *= i; sum += temp; }
```

5.2 循环控制语句

| 语句       | 作用               | 关键点   |
|----------|------------------|-------|
| break    | 跳出当前循环           | 只跳出一层 |
| continue | 跳过本次循环剩余部分，进入下一轮 | 不跳出循环 |

5.3 循环嵌套

- 外层循环一次，内层循环全部执行完
- 内层 break 只跳出内层，不影响外层

5.4 本章习题

| 题号   | 题型                           |
|------|------------------------------|
| 5-1  | 流程图：3个程序段的流程图                |
| 5-2  | 补充程序并统计循环次数                  |
| 5-3  | 编程：最大公约数和最小公倍数               |
| 5-4  | 编程：统计字符（字母/空格/数字/其他）         |
| 5-5  | 编程：Sn = a+aa+aaa+...         |
| 5-6  | 编程：1!+2!+...+20!             |
| 5-7  | 编程：三组求和式的总和                  |
| 5-8  | 编程：水仙花数                      |
| 5-9  | 编程：完数（含因子输出）                 |
| 5-10 | 编程：分数序列 2/1+3/2+5/3+... 前20项 |
| 5-11 | 编程：球弹跳 100m 自由落体             |

| 题号   | 题型                 |
|------|--------------------|
| 5-12 | 编程：猴子吃桃（倒推法）       |
| 5-13 | 编程：迭代法求 $\sqrt{a}$ |
| 5-14 | 编程：牛顿迭代法求方程根       |
| 5-15 | 编程：二分法求方程根         |
| 5-16 | 编程：输出菱形图案          |
| 5-17 | 编程：乒乓球比赛名单         |

## 5.5 本章错题复盘

### ⚠ 5-4 统计字符 — fgets 陷阱

**错误：**`while(str[i]!='\0')` 把 `\n` (fgets保留的换行) 计入了 other

**修正：**`while(str[i]!='\0' && str[i]!='\n')`

### ⚠ 5-7 三组求和 — 循环范围串用

**错误：**`if(i<=50) sum3+=1.0/i` 把 sum3 (应只到10) 绑到了 50

**修正：**sum3 单独用 `if(i<=10)` 控制

### ⚠ 5-11 球弹跳 — 变量含义混淆

**错误：**代码中 `h` 实际存的是 **2×反弹高度**，直接输出 `h` 当作反弹高度，多了一倍

**修正：**输出 `h/2.0`

### 🔥 5-6 阶乘和 — 数据类型（自总结很到位）

- `int` 存不下 20! (溢出) → 用 `unsigned long long`
- `%d` 打不了 → 用 `%llu`
- 循环内重复声明 `int i` → 编译错误

### 🔥 5-13 迭代法 — do-while vs while

- `while` + 假初值骗循环可能只跑一轮就退出
- 迭代类问题 `do-while` 更自然：先算再判断

## 5.6 经典题型速查

| 题型    | 核心思路                                                                               |
|-------|------------------------------------------------------------------------------------|
| 最大公约数 | 辗转相除 <code>while(b!=0){t=a%b;a=b;b=t;}</code>                                      |
| 水仙花数  | 三位拆分：百= <code>i/100</code> , 十= <code>i/10%10</code> , 个= <code>i%10</code> , 判立方和 |
| 完数    | 因子只查到 <code>i/2</code> ，双遍扫描（先求和再输出）                                               |
| 分数序列  | 分子分母是相邻斐波那契数： <code>next=a+b; b=a; a=next</code>                                   |

| 题型   | 核心思路                                                     |
|------|----------------------------------------------------------|
| 猴子吃桃 | 倒推: $n = (n+1)*2$ , 从第10天倒推9次                            |
| 牛顿迭代 | $x = x - f(x)/f'(x)$ , do-while + fabs                   |
| 二分法  | 中点 $(L+R)/2$ , $f(mid)<0 \rightarrow L=mid$ , 否则 $R=mid$ |
| 菱形图案 | 对称公式: $stars = n - 2*abs(i - mid)$                       |

## 5.7 易错清单

1. `fabs()` 用于浮点绝对值, `abs()` 用于整数 — 用错无报错但结果错
2. `do-while` 末尾分号不能丢
3. 整除陷阱:  $4/3 = 1$ , 应该写  $4.0/3.0$
4. 循环变量不更新  $\rightarrow$  死循环
5. `float` 精度有限, 迭代类问题优先用 `double`
6. 数据太大用 `unsigned long long` (阶乘、幂次等)

# 第6章 利用数组处理批量数据

## 6.1 一维数组

```
int a[5] = {1,2,3,4,5};    // 全初始化
int a[5] = {1,2};         // 部分初始化  $\rightarrow$  {1,2,0,0,0}
int a[] = {1,2,3};        // 不指定长度  $\rightarrow$  长度3
int a[5] = {0};           // 全部置0
// ⚠️ 下标从0开始: a[0] 到 a[4]
```

## 6.2 二维数组

```
int a[3][3] = {{1,2,3}, {4,5,6}, {7,8,9}};
int a[][3] = {{1,2,3}, {4,5,6}};           // 第一维可省, 第二维必须写!
// a[i][j] - i行j列, 行优先存储
```

## 6.3 字符数组与字符串

```
char s[] = "hello";          // 自动加\0, 长度6
char s[5] = {'h','e'};      // 不是字符串! 缺\0

// 输入
gets(s);                    // 到换行止, 丢弃\n, 加\0 (❌不安全)
fgets(s, 50, stdin);        // 安全, 但保留\n
scanf("%s", s);             // 遇空白止, 自动加\0
```

```
// 输出
puts(s);           // 自动换行
printf("%s", s);    // 不自动换行
```

三种输入对比

| 方法               | 遇空格 | 安全   | 换行处理  |
|------------------|-----|------|-------|
| scanf("%s")      | 停   | ✗    | 残留缓冲区 |
| gets()           | 不停  | ✗ 危险 | 丢弃\n  |
| fgets(s,n,stdin) | 不停  | ✓ 安全 | 保留\n  |

6.4 本章习题

| 题号   | 题型              |
|------|-----------------|
| 6-1  | 筛选法求100内素数      |
| 6-2  | 选择法对10个整数排序     |
| 6-3  | 3×3矩阵对角线元素之和    |
| 6-4  | 有序数组插入一个数       |
| 6-5  | 数组逆序重新存放        |
| 6-6  | 杨辉三角（10行）       |
| 6-7  | 魔方阵（1~n²）       |
| 6-8  | 找出二维数组中的鞍点      |
| 6-9  | 折半查找法           |
| 6-10 | 3行80字符统计各类字符    |
| 6-11 | 输出特定星号图案        |
| 6-12 | 密码翻译（Atbash）    |
| 6-13 | 字符串连接（不用strcat） |
| 6-14 | 字符串比较（不用strcmp） |
| 6-15 | 字符串复制（不用strcpy） |

6.5 本章错题复盘

⚠ 6-14 字符串比较 — 相等输出不一致

错误：相等时 printf("相等\n")

修正：题意要求 s1==s2 时输出 0

6.6 经典算法速查

| 算法    | 核心代码                                                                                  |
|-------|---------------------------------------------------------------------------------------|
| 筛选法素数 | <code>for(i=2;i*i&lt;=100;i++) if(a[i]) for(j=i*i;j&lt;=100;j+=i) a[j]=0;</code>      |
| 选择排序  | <code>min=i; for(j=i+1;j&lt;n;j++) if(a[j]&lt;a[min])min=j; swap(a[i],a[min]);</code> |
| 冒泡排序  | <code>for(i=0;i&lt;n-1;i++) for(j=0;j&lt;n-1-i;j++) if(a[j]&gt;a[j+1])swap;</code>    |
| 有序插入  | 找位→后移→插入。边界：比所有大时直接放末尾                                                                |
| 逆序    | <code>for(i=0;i&lt;n/2;i++) swap(a[i],a[n-1-i]);</code>                               |
| 折半查找  | 升序: <code>a[m]&lt;x→l=m+1</code> ; 降序: <code>a[m]&gt;x→l=m+1</code>                   |
| 杨辉三角  | <code>a[i][0]=a[i][i]=1; a[i][j]=a[i-1][j-1]+a[i-1][j];</code>                        |
| 鞍点    | 行最大→该列最小→判相等                                                                          |
| 魔方阵   | 1放首行中。右上为空→去右上, 被占→去正下方                                                               |
| 密码翻译  | <code>'A'+'Z'-c</code> = 反向字母 (Atbash密码)                                              |

## 6.7 手写字符串函数

| 函数     | 核心实现                                           |
|--------|------------------------------------------------|
| strlen | <code>while(*p) i++,p++; return i;</code>      |
| strcpy | <code>while(*s2) *s1++=*s2++; *s1='\0';</code> |
| strcat | 先找到s1的\0, 再copy s2                             |
| strcmp | 逐字比较, 遇不等返回差值; 都到\0返回0                         |

## 6.8 易错清单

1. 数组下标越界不报错! `a[10]` 访问的是第11个位置
2. `scanf("%s")` 遇空格就停, 读不了带空格的字符串
3. 字符串必须 `\0` 结尾, 忘了写会输出乱码
4. `gets()` 不安全但有考题, 注意它丢弃 `\n`
5. `fgets()` 保留 `\n`, 需要时手动去掉
6. 二维数组作参数: `int a[][N]` 第二维必须指定。原因是 C 语言采用行优先连续存储, 编译器必须知道列数 N 才能通过公式 `地址 = 首地址 + (i × N + j) × sizeof(元素类型)` 计算出第 i 行第 j 列的偏移量, 否则无法完成二维寻址
7. 折半查找要求数组已排序, 升序降序的方向不同
8. 鞍点: 同行可能有多个相同最大值, 只取第一个可能漏检

# 第7章 用函数实现模块化程序设计

## 7.1 函数定义

```
返回类型 函数名(参数列表)    // 没有返回值用 void
{
    函数体;
    return 值;                // void 函数可省略 return
}
```

## 7.2 函数声明 vs 定义

```
// 声明（告诉编译器有这个函数）- 放 main 之前或头文件
int gcd(int a, int b);

// 定义（函数体）- 可以放 main 之后
int gcd(int a, int b) { ... }
```

## 7.3 参数传递

```
void swap(int x, int y) { ... }    // 值传递：不影响实参
void swap(int *p1, int *p2) { ... } // 地址传递：影响实参（第8章详细讲）

// 数组作参数（传递的是地址）
void sort(int a[], int n) { ... }  // 修改数组会影响实参！
```

## 7.4 局部变量 vs 全局变量

|      | 局部变量   | 全局变量     |
|------|--------|----------|
| 作用域  | 函数内部   | 整个文件     |
| 生命周期 | 函数执行期间 | 程序运行全程   |
| 可读性  | ✅ 好    | ⚠️ 破坏模块化 |

## 7.5 递归

```
// 勒让德多项式 (7-13)
float p(int n, float x) {
    if (n == 0) return 1;        // 基案1
    if (n == 1) return x;        // 基案2
    return ((2*n-1)*x*p(n-1,x) - (n-1)*p(n-2,x)) / n;
}

// 整数转字符串 (7-17) - 先递归再输出 = 正序
void convert(int n) {
    if (n < 0) { putchar('-'); n = -n; }
    if (n / 10) convert(n / 10); // 递归剥离
}
```

```
    putchar(n % 10 + '0');        // 回溯输出
}
```

递归三要素：①基案（终止条件）②递归公式（问题缩小）③回溯时机（先递归→逆序，后递归→正序）

## 7.6 函数指针（进阶）

```
// 7-13: 矩形法求定积分 - 传什么函数就算什么积分
double integral(double (*f)(double), double a, double b, int n) {
    double h = (b-a)/n, sum = 0;
    for (int i = 0; i < n; i++)
        sum += f(a + i*h) * h;
    return sum;
}
// 调用: integral(sin, 0, 1, 100000) → ∫01 sin(x)dx
//       integral(cos, 0, 1, 100000) → ∫01 cos(x)dx
//       integral(exp, 0, 1, 100000) → ∫01 exdx
```

## 7.7 结构体 + 函数（7-15）

```
struct Employee { int id; char name[20]; };

void sort(struct Employee emp[], int n) {
    // 结构体整体赋值交换 → 姓名随职工号一起动
    struct Employee temp = emp[j];
    emp[j] = emp[j+1];
    emp[j+1] = temp;
}

void search(struct Employee emp[], int n, int target) {
    // 二分查找: mid = low + (high-low)/2 ← 防溢出写法
    while (low <= high) { ... }
}
```

## 7.8 本章习题

| 题号 | 题型 |
|----|----|
|----|----|

|     |                    |
|-----|--------------------|
| 7-1 | 两个函数：最大公约数 + 最小公倍数 |
|-----|--------------------|

|     |                                     |
|-----|-------------------------------------|
| 7-2 | 三个函数：一元二次方程 $>0$ / $=0$ / $<0$ 三种情况 |
|-----|-------------------------------------|

|     |        |
|-----|--------|
| 7-3 | 素数判断函数 |
|-----|--------|

|     |           |
|-----|-----------|
| 7-4 | 3×3矩阵转置函数 |
|-----|-----------|

|     |           |
|-----|-----------|
| 7-5 | 字符串反序存放函数 |
|-----|-----------|

| 题号   | 题型                        |
|------|---------------------------|
| 7-6  | 两字符串连接函数                  |
| 7-7  | 元音字母复制到另一字符串              |
| 7-8  | 4位数字逐位输出（加空格）             |
| 7-9  | 统计字符串中字母/数字/空格/其他         |
| 7-10 | 输出最长单词                    |
| 7-11 | 冒泡排序10个字符                 |
| 7-12 | 牛顿迭代法求方程根（函数）             |
| 7-13 | 递归求勒让德多项式                 |
| 7-14 | 成绩统计四函数（学生均分/课程均分/最高分/方差） |
| 7-15 | 职工信息：结构体+排序+折半查找          |
| 7-16 | 十六进制转十进制函数                |
| 7-17 | 递归整数转字符串                  |
| 7-18 | 年月日计算是該年第几天               |

## 7.9 本章错题复盘

### ⚠ 7-2 一元二次方程 — $\Delta < 0$ 处理

错误：`printf("无解\n")`

修正： $\Delta < 0$  有两共轭复根，应输出

```
float real = -b/(2*a);
float imag = sqrt(-dt)/(2*a);
printf("x1=%.3f+%.3fi, x2=%.3f-%.3fi\n", real, imag, real, imag);
```

### ⚠ 7-6 字符串连接 — 局部数组陷阱

错误：函数内部定义 `char str3[50]`，拼接后在函数内 `printf`，调用方拿不到结果

修正：传入结果数组作为参数 `void connect(char s1[],char s2[],char result[])`

### ⚠ 7-8 四位数字输出 — 尾随空格

错误：`printf("%d ", ...)` 每个数字后一个空格，最后多一个

修正：`printf("%d %d %d %d\n", a,b,c,d)` 一次输出

### 🔥 7-10 最长单词 — 内容未保存

错误：只记录了最长长度，没保存最长单词的内容；最后一个单词（无空格结尾）没比较

修正：记录最长单词的起始位置 `max_start`，输出时从该位置取 `max_len` 个字符

## 7.10 易错清单

- 1. 函数内定义的局部数组，返回后内存被回收（别返回局部数组地址）
- 2. 数组作参数不需 `&`，函数内修改会影响原数组
- 3. 递归忘记终止条件 → 栈溢出
- 4. `void` 函数不需要 `return` 语句
- 5. 全局变量破坏模块化，尽量用参数传递
- 6. 函数声明末尾要分号，定义末尾不要分号

# 第8章 善于利用指针

## 8.1 指针即地址

```
int a = 10;
int *p = &a;      // p 存的是 a 的地址，*p 是 a 的值
printf("%d", *p); // 10 - 解引用
```

## 8.2 指针类型速查

| 写法                               | 含义                      | 典型场景                       |
|----------------------------------|-------------------------|----------------------------|
| <code>int *p</code>              | 指向 <code>int</code> 的指针 | 变量交换                       |
| <code>int **pp</code>            | 指向指针的指针（二级）             | 指针数组排序                     |
| <code>int (*p)[N]</code>         | 指向数组的指针                 | 二维数组参数                     |
| <code>int *p[N]</code>           | 指针数组（N个指针）              | 不等长字符串排序                   |
| <code>double (*f)(double)</code> | 函数指针                    | 传 <code>sin/cos/exp</code> |

## 8.3 指针运算

```
int a[] = {1,2,3,4,5};
int *p = a;      // 或 int *p = &a[0];

// 四种等价写法访问 a[i]:
a[i] == *(a + i) == p[i] == *(p + i)

// p++ 移向下一个元素（自动跳过 sizeof(int) 字节）
// 二维: a[i][j] == (*(a + i) + j)
```

## 8.4 指针与函数

值传递 vs 地址传递

```
void swap_value(int x, int y) { int t=x; x=y; y=t; } // 不影响实参
void swap_ptr(int *p1, int *p2) { int t=*p1; *p1=*p2; *p2=t; } // 影响实参
```

数组作参数（自动传地址）

```
void sort(int *arr, int n) { ... } // 等价于 int arr[]
void transpose(int (*a)[3]) { ... } // 二维数组参数
```

函数指针作参数

```
double integral(double (*f)(double), double a, double b, int n);
// 第一个参数是函数指针
```

8.5 动态内存

```
int *a = (int *)malloc(n * sizeof(int));
// ... 使用 ...
free(a); // ⚠️ 必须释放！
```

8.6 本章习题

| 题号   | 题型                      |
|------|-------------------------|
| 8-1  | 指针：3个整数排序               |
| 8-2  | 指针数组：3个字符串排序            |
| 8-3  | 指针：10个数，最小→第一，最大→最后     |
| 8-4  | 指针：n个数循环后移m个位置          |
| 8-5  | 约瑟夫环（n人报数到3退出）          |
| 8-6  | 指针：求字符串长度函数             |
| 8-7  | 指针：从第m个字符开始复制           |
| 8-8  | 指针：统计字符（大写/小写/空格/数字/其他） |
| 8-9  | 指针：3×3矩阵转置              |
| 8-10 | 指针：5×5矩阵极值重排            |
| 8-11 | 指针数组：10个等长字符串排序         |
| 8-12 | 指针数组+动态分配：不等长字符串排序      |

| 题号   | 题型                   |
|------|----------------------|
| 8-13 | 函数指针：矩形法求定积分         |
| 8-14 | 指针：n个数逆序排列           |
| 8-15 | 指针：学生成绩统计（均分/不及格/优秀） |
| 8-16 | 指针：提取字符串中的整数         |
| 8-17 | 指针：自定义strcmp函数       |
| 8-18 | 指针数组：月份号→英文月名        |
| 8-19 | 模拟malloc/free内存分配器   |
| 8-20 | 二级指针：5个字符串排序         |
| 8-21 | 二级指针：n个整数排序          |

8.7 本章错题复盘

⚠ 8-6 字符串长度 — 未按要求封装

错误：把求长度逻辑写在 main 里，题目要求"写一函数"

修正：提取 `int str_len(char *p)` 函数

🔥 8-15 第1门课平均分 — 指针解引用错误

错误：`sum += **(score)` — `**(score)` 等价于 `score[0][0]`，循环4次加了同一个值

修正：`sum += (*(score + i))` — 等价于 `score[i][0]`，取每个学生的第1门课

⚠ 8-18 月份英文名 — 缺输入校验

错误：`scanf("%d",&n); puts(*(months+n-1))` — `n=13` 时越界

修正：`if(n>=1 && n<=12) puts(...); else printf("输入错误");`

8.8 指针经典用法速查

| 题型     | 形参写法                             | 核心技巧                                                  |
|--------|----------------------------------|-------------------------------------------------------|
| 变量交换   | <code>int *p1, *p2</code>        | swap 交换 <code>*p1</code> 和 <code>*p2</code>           |
| 一维数组   | <code>int *arr</code>            | <code>*(arr+i)</code> 等价 <code>arr[i]</code>          |
| 二维数组   | <code>int (*a)[5]</code>         | <code>*(*(a+i)+j)</code> 等价 <code>a[i][j]</code>      |
| 字符串遍历  | <code>char *p</code>             | <code>while(*p) ...</code> ，遇 <code>\0</code> 停止      |
| 指针数组排序 | <code>char *p[]</code>           | 只交换指针，不挪字符串                                           |
| 二级指针   | <code>int **p</code>             | 比较 <code>**(p+i)</code> ，交换 <code>*(p+i)</code>       |
| 函数指针   | <code>double (*f)(double)</code> | 传函数名作参数                                               |
| 动态内存   | <code>malloc/free</code>         | <code>int *a = malloc(n*sizeof(int)); free(a);</code> |

## 8.9 易错清单

- 1. 指针没初始化就解引用 → 野指针，程序崩溃
- 2. `scanf("%d", p)` 不需要 `&` (`p` 本身就是地址)
- 3. `int *p` 的 `*` (定义) 和 `*p` 的 `*` (解引用) 含义不同
- 4. `*(a + i)` 和 `*a + i` 完全不同：前者=`a[i]`，后者=`a[0]+i`
- 5. 指针数组排序只交换指针，不交换字符串本体
- 6. `malloc` 后必须 `free`，否则内存泄漏
- 7. 数组名是指针常量：`a = p` 非法，`p = a` 合法
- 8. 二级指针 `**p` 做参数时，`sizeof` 和数组维度容易搞混

# 附录：C语言输入输出速查手册

## A.1 输出函数总览

| 函数                   | 头文件                          | 用途         | 示例                              |
|----------------------|------------------------------|------------|---------------------------------|
| <code>printf</code>  | <code>&lt;stdio.h&gt;</code> | 格式化输出      | <code>printf("a=%d", a);</code> |
| <code>putchar</code> | <code>&lt;stdio.h&gt;</code> | 输出单个字符     | <code>putchar('A');</code>      |
| <code>puts</code>    | <code>&lt;stdio.h&gt;</code> | 输出字符串并自动换行 | <code>puts("Hello");</code>     |

## A.2 printf 格式说明符

格式：`%[标志][宽度][.精度]类型字符`

### 基本类型说明符（全13种）

| 说明符              | 含义                                    | 示例                                   | 输出           |
|------------------|---------------------------------------|--------------------------------------|--------------|
| <code>%d</code>  | 有符号十进制整数                              | <code>printf("%d", 97);</code>       | 97           |
| <code>%u</code>  | 无符号十进制整数                              | <code>printf("%u", -1);</code>       | 4294967295   |
| <code>%ld</code> | 长整型                                   | <code>printf("%ld", 100000L);</code> | 100000       |
| <code>%f</code>  | 单双精度浮点数                               | <code>printf("%f", 3.14);</code>     | 3.140000     |
| <code>%lf</code> | 双精度（推荐一律用 <code>%f</code> ）           | <code>printf("%lf", 3.14);</code>    | 3.140000     |
| <code>%e</code>  | 科学计数法（小写e）                            | <code>printf("%e", 314.0);</code>    | 3.140000e+02 |
| <code>%E</code>  | 科学计数法（大写E）                            | <code>printf("%E", 314.0);</code>    | 3.140000E+02 |
| <code>%g</code>  | 自动选 <code>%f</code> 或 <code>%e</code> | <code>printf("%g", 3.14);</code>     | 3.14         |
| <code>%c</code>  | 单个字符                                  | <code>printf("%c", 65);</code>       | A            |
| <code>%s</code>  | 字符串                                   | <code>printf("%s", "abc");</code>    | abc          |
| <code>%o</code>  | 无符号八进制                                | <code>printf("%o", 10);</code>       | 12           |

| 说明符                | 含义      | 示例                                 | 输出                    |
|--------------------|---------|------------------------------------|-----------------------|
| <code>%x/%X</code> | 无符号十六进制 | <code>printf("%x", 255);</code>    | <code>ff</code>       |
| <code>%p</code>    | 指针（地址）  | <code>printf("%p", &amp;a);</code> | <code>0061FF1C</code> |
| <code>%%</code>    | 输出百分号本身 | <code>printf("100%%");</code>      | <code>100%</code>     |

记忆口诀：d整 f浮 c字符 s串 / o八 x十六 e科学 / u无符 ld长整 %本身

宽度控制

| 格式                | 含义        | 输出                 |
|-------------------|-----------|--------------------|
| <code>%5d</code>  | 至少占5列，右对齐 | <code>42</code>    |
| <code>%-5d</code> | 至少占5列，左对齐 | <code>42</code>    |
| <code>%05d</code> | 不足5列用0填充  | <code>00042</code> |

精度控制（浮点数）

| 格式                  | 含义          | 输出                 |
|---------------------|-------------|--------------------|
| <code>%.2f</code>   | 保留2位小数      | <code>3.14</code>  |
| <code>%.0f</code>   | 不保留小数（四舍五入） | <code>4</code>     |
| <code>%8.2f</code>  | 总宽8列，2位小数   | <code>3.14</code>  |
| <code>%10.3f</code> | 总宽10列，3位小数  | <code>1.500</code> |

字符串截断

| 格式                 | 含义          | 输出               |
|--------------------|-------------|------------------|
| <code>%.3s</code>  | 只输出前3个字符    | <code>Hel</code> |
| <code>%8.3s</code> | 8列宽，只显示前3字符 | <code>Hel</code> |

A.3 输入函数总览

| 函数                   | 头文件                          | 用途      | 示例                                    |
|----------------------|------------------------------|---------|---------------------------------------|
| <code>scanf</code>   | <code>&lt;stdio.h&gt;</code> | 格式化输入   | <code>scanf("%d", &amp;a);</code>     |
| <code>getchar</code> | <code>&lt;stdio.h&gt;</code> | 读取单个字符  | <code>c = getchar();</code>           |
| <code>gets</code>    | <code>&lt;stdio.h&gt;</code> | 读取一行字符串 | <code>gets(str);</code> ⚠️ 不安全已过时     |
| <code>fgets</code>   | <code>&lt;stdio.h&gt;</code> | 安全地读取一行 | <code>fgets(str, size, stdin);</code> |

A.4 scanf 格式说明符

```
scanf("格式串", &变量1, &变量2, ...);  
//          ↑ 注意：必须加取地址符 &
```

| 说明符 | 含义              | 示例                         |
|-----|-----------------|----------------------------|
| %d  | 读入十进制整数         | scanf("%d", &a);           |
| %ld | 读入长整型           | scanf("%ld", &a);          |
| %f  | 读入 float 浮点数    | scanf("%f", &x);           |
| %lf | 读入 double 浮点数   | scanf("%lf", &y); ⚠️ 区分于%f |
| %c  | 读入一个字符 (⚠️ 含空格) | scanf("%c", &c);           |
| %s  | 读入字符串 (到空白为止)   | scanf("%s", str); 不加&      |

格式串中的普通字符

```
scanf("a=%d,b=%d", &a, &b);  
//      ^^  ^^  这些普通字符必须原样输入！  
//  正确输入：a=3,b=7  
//  错误输入：3 7  ← 缺少 a= 和 ,
```

连续输入

```
// 数值连续输入 - 空格/回车/Tab 自动分割  
scanf("%d%d%d", &a, &b, &c); // 输入：3 7 5 或 3换行7换行5  
  
// 字符连续输入 - 无空白分割  
scanf("%c%c", &c1, &c2); // 输入：AB (中间不能有空格！)
```

A.5 scanf 四大陷阱 (★ 考试最高频)

陷阱一：%c 不跳过空白字符

```
scanf("%d", &n);  
scanf("%c", &ch);  
// 输入：123换行A  
// 结果：n=123, ch='\n' (读到了换行符，不是'A'！)
```

解决方案：

```
// 方案1: 在 %c 前加空格, 让 scanf 跳过所有空白
scanf(" %c", &ch); // 注意 %c 前面有个空格!

// 方案2: 用 getchar() 吃掉缓冲区残留
scanf("%d", &n);
getchar();          // 吃掉换行符
scanf("%c", &ch);
```

## 陷阱二：格式串中的普通字符必须原样输入

```
scanf("x=%f,y=%f", &x, &y);
// 必须输入: x=8.5,y=71.82 ← 字母和逗号一个不能少
```

## 陷阱三：scanf 到 %s 不读空格

```
char str[20];
scanf("%s", str);
// 输入: Hello World
// 结果: str = "Hello" (World 留在缓冲区)
```

## 陷阱四：浮点类型混淆

```
float x;   scanf("%f", &x); // 正确: 用 %f
double y;  scanf("%lf", &y); // 正确: 用 %lf ← 区分!

// printf 输出: 现代 C 标准中 printf 打印 double 可用 %f 或 %lf
// 但为考试绝对安全且兼容老旧编译器, 建议牢记:
// 输入 (scanf) 必须区分: float 用 %f, double 用 %lf
// 输出 (printf) 一律用 %f (float 和 double 都用 %f)
printf("%f", x); // 正确
printf("%f", y); // 正确 (printf 对所有浮点类型一律用 %f)
```

## A.6 字符输入输出

### putchar

```
putchar('A'); // 输出: A
putchar(65);  // 输出: A (ASCII码65='A')
putchar('\n'); // 输出换行
putchar(c1);   // 输出变量 c1 对应的字符
```

## getchar

```
char c;  
c = getchar();           // 从键盘读入一个字符（含空格/换行）
```

### 典型用途：吃换行符

```
scanf("%d", &n);  
getchar();               // 吃掉 scanf 遗留的换行符  
c = getchar();           // 现在读到的是真正的字符
```

## 字符与 ASCII 码互转

```
char c = 'A';  
printf("%c", c);         // 按字符输出 → A  
printf("%d", c);         // 按ASCII码输出 → 65  
  
int n = 97;  
printf("%c", n);         // 按字符输出 → a  
printf("%d", n);         // 按整数输出 → 97
```

## A.7 字符串输入输出

### puts — 输出字符串

```
char str[] = "Hello";  
puts(str);             // 输出 Hello 并自动换行  
puts("World");         // 输出 World 并自动换行
```

### gets — 读取一行（✗ 考试可能还在考，但已淘汰）

```
char str[50];  
gets(str);             // 读到换行符，丢弃换行，\0 结尾  
// ⚠ 危险：不检查数组长度，可能越界！
```

### fgets — 安全替代

```
fgets(str, 50, stdin); // 最多读49字符，保留\0位置  
// ⚠ 注意：fgets 会保留换行符 \n 在字符串末尾
```

```
// 去掉fgets末尾的\n
str[strcspn(str, "\n")] = '\0';
```

A.8 标准易错点汇总

| 易错点         | 错误写法               | 正确写法                  |
|-------------|--------------------|-----------------------|
| scanf 缺 &   | scanf("%d", a);    | scanf("%d", &a);      |
| printf 加 &  | printf("%d", &a);  | printf("%d", a);      |
| %c 吃空格      | scanf("%c%c")输入A B | scanf(" %c %c")       |
| 浮点输入用错      | scanf("%d", &x);   | scanf("%f", &x);      |
| printf 精度误用 | printf("%.2d", 5); | printf("%.2f", 3.14); |
| double 输入   | scanf("%f", &y);   | scanf("%lf", &y);     |
| 格式串有普通字符    | 只输入数据              | 必须原样输入                |

A.9 期末考试常见题型

题型一：写输出结果

```
int a = 5, b = 7;
printf("a=%d,b=%d\n", a, b);
// 输出：a=5,b=7
```

题型二：字符与ASCII互转

```
char c = 'A';
printf("%c %d\n", c, c);
// 输出：A 65
```

题型三：格式控制

```
printf("%5d\n", 123); // 输出： 123
printf("%05d\n", 123); // 输出：00123
printf("%.2f\n", 3.1); // 输出：3.10
printf("%-5d\n", 123); // 输出：123
```

题型四：scanf 陷阱分析

```
int n; char c;
scanf("%d", &n);
scanf("%c", &c);
// 输入: 10换行A
// 问: c 读到了什么?
// 答: '\n' (换行符), 不是 'A'
```

### 题型五: 格式串中的普通字符

```
scanf("a=%db=%d", &a, &b);
// 应如何输入使 a=3, b=7?
// 答: a=3b=7
```

### 题型六: char 溢出

```
char c = 197;           // 超出 signed char 范围
printf("%d\n", c);
// 输出: -59 (发生溢出)
```

### 题型七: 短路求值

```
int a = 3, b = 4;
!(a > b) && !(c = 5) && 0;
// 问: c 的值?
// 答: c 未被赋值, 因为 && 短路 - 左边为假, 右边不执行
```

## ■ 全书复习路线图

第3章(输入输出)→第4章(选择)→第5章(循环) 是**基础铁三角**

第6章(数组)→第7章(函数)→第8章(指针) 是**重点加分项**

**编程题多练:** 数组排序、字符串处理、指针操作、递归

**读程序题重点抓:** scanf陷阱、char/ASCII转换、短路求值、循环边界、指针运算

**简答题背记:** 算法五大特性、三种基本结构、结构化程序设计、C语言特点